

기술 보고서

웹 크롤링 프로젝트 테크니컬 보고서

Web Crawling Project — Technical Report

과 목: AI 서비스 프로그래밍

학 번: 22503110159

이 름: 김광진

서론

배경

영화 리뷰 데이터를 활용한 추천 시스템을 프로젝트 기간 동안 두 차례에 걸쳐 설계·구현했다. 처음에는 BGE-M3 임베딩과 FAISS 벡터 검색, GRU 감성 분류기를 결합한 의미 기반 추천 서비스를 만들었다. 이후 "GRU 감성 필터가 벡터 검색 결과와 실질적으로 중복된다"는 피드백을 계기로 한계를 재검토했고, 모델이 서비스 동작에 실질적으로 기여하는 새로운 구조로 방향을 전환해 LSTM 기반 키워드 감성 추천 서비스를 완성했다.

목적

본 보고서는 두 서비스의 설계·구현·평가 과정을 시간 순서대로 기록하고, 기존 서비스에서 발견된 한계가 무엇이었는지, 그 한계가 현재 서비스의 설계 결정에 어떻게 반영되었는지를 명확히 밝히는 것을 목적으로 한다.

보고서 구성

부	제목	내용
1 부	기존 서비스 (BGE-M3 + FAISS + GRU)	데이터 수집, 전처리, 벡터 인덱스 구축, GRU 감성 분류기, 서비스 설계 및 성능 평가
2 부	개선 배경	기존 서비스의 한계 발견, 피드백, 새 요구사항
3 부	현재 서비스 (LSTM 키워드 감성 추천)	방향 전환 과정, LSTM 이진 분류기 학습, 키워드 인덱스 + Wilson 랭킹, 서비스 설계, 검증 및 한계

1 부 · 기존 서비스

BGE-M3 + FAISS + GRU

의미 기반 리뷰 검색과 GRU 감성 필터를 결합한 첫 번째 추천 서비스

1. 데이터 수집

1.1 수집 대상 및 도구

Letterboxd 에서 영화 목록과 리뷰를, IMDB 에서 사람이 라벨링한 검증용 리뷰를 수집했다. Letterboxd 는 Cloudflare 로 보호되어 있어 일반 requests 로는 차단된다.

- 영화 목록: undetected-chromedriver — 실제 브라우저처럼 동작해 Cloudflare 우회
- 리뷰 본문: curl_cffi AsyncSession — 브라우저 TLS 핑거프린트를 흉내내어 비동기 대량 요청

1.2 수집 결과

항목	값
최종 리뷰 수	40,217,722 건
영화 수	26,761 편
통합 CSV 크기	약 12GB

1.3 수집 과정에서 발생한 문제

스크롤 자동화 및 영화 이름 통일

무한 스크롤 페이지에서 누락 없이 전량을 수집하기 위한 자동 스크롤 로직을 구현했고, 같은 영화가 사이트별로 다른 표기로 등록된 경우를 통일해 중복 집계를 방지했다.

짧은 리뷰 및 반복 텍스트 필터

의미 없는 짧은 리뷰와 반복적인 텍스트를 걸러내기 위한 not_repetitive() 함수의 임계값이 지나치게 엄격(0.3)하게 설정되어 영어 리뷰의 99.3%가 잘못 제거되는 문제가 있었다. 임계값을 0.1 로 완화해 해결했다.

파일 잠금 버그

중간 산출물인 combined.csv 를 Excel(PID 26256)이 열어둔 상태에서 재병합 스크립트가 실행되어 파일 잠금 충돌이 발생했다. Excel 프로세스를 종료한 뒤 PowerShell 로 재병합해 2,449MB 였던 파일을 12,998MB 로 정상 복구했다.

2. 데이터 전처리

수집된 원본 CSV 는 크롤링 단계의 노이즈(중복, 반복 텍스트, 형식 불일치)를 포함하고 있어 임베딩·학습 이전에 정제 단계를 거쳤다.

- 평점 정규화: 사이트별 표기(별점 등)를 10 점 척도로 통일
- 중복 제거 및 영화명 통일: 동일 영화의 이형 표기를 하나의 키로 병합
- 저품질 리뷰 필터링: `not_repetitive()` 임계값 조정(0.1)으로 반복·저정보 리뷰 제거
- 결과: 정제된 CSV 를 이후 임베딩·학습 파이프라인의 입력으로 사용

3. BGE-M3 임베딩 + FAISS 인덱스 구축

3.1 임베딩 모델

BGE-M3 (BAAI/bge-m3)를 사용했다. 다국어 트랜스포머 기반 모델로 한/영 혼합 리뷰를 단일 모델로 처리할 수 있다.

```
model = BGEM3FlagModel("BAAI/bge-m3", use_fp16=True, device="cuda")
# 256 차원으로 슬라이싱 (원래 1024 차원)
vecs = model.encode(texts, batch_size=128, max_length=512,
                    return_dense=True)["dense_vecs"][:, :256]
```

3.2 FAISS 인덱스 구조 (IVF-PQ)

```
quantizer = faiss.IndexFlatIP(DIM) # 내적 기반 거리
index_review = faiss.IndexIVFPQ(quantizer, 256, 8192, 16, 8)
# NLIST=8192, M_PQ=16, NBITS=8, 검색 시 nprobe=64
```

IVF(Inverted File Index)는 벡터를 8,192 개 클러스터로 나눠 검색 시 일부 클러스터만 탐색한다. PQ(Product Quantization)는 각 서브벡터를 압축해 메모리 사용량을 줄인다. 검색 단계에서는 stage1_k(1 차 후보 수)를 100→300 으로, stage2_k(재순위 후보 수)를 2000→5000 으로 늘려 재현율을 튜닝했다.

3.3 구현 중 발생한 문제

RAM 누적 문제

영화별 리뷰 통계(review_meta)를 메모리에 전량 누적하는 방식은 4 천만 건 규모에서 메모리 부족을 일으켰다. review_meta 를 TSV 파일로 스트리밍 저장하는 방식으로 전환해 해결했다.

GPU 유틸 문제

CSV 를 iterrows()로 순차 읽으면 GPU 가 데이터 대기 상태로 유틸 시간이 길어졌다. threading + queue 기반의 producer-consumer 구조인 csv_prefetch_iter()를 도입해 CSV 읽기와 GPU 인코딩을 병렬화했다.

4. GRU 감성 분류기

4.1 모델 구조

```
Sequential([
    Embedding(40001, 32, input_length=100), # 임베딩 32 차원
    GRU(64),                                # GRU 유닛 64 개
    Dense(16, activation="tanh"),
    Dense(2, activation="softmax")
])
# optimizer: RMSprop(lr=0.001)
# loss: categorical_crossentropy
# EarlyStopping(patience=3)
# MAX_LEN=100, VOCAB_SIZE=40000
```

4.2 학습 데이터 및 라벨링

평점 7~8 점 리뷰(긍정)와 3~4 점 리뷰(부정)를 각 50 만 건씩 사용했다. 9~10 점과 1~2 점 극단값은 트롤 리뷰 가능성이 있어 학습에서 제외했다.

4.3 3 단계 학습 과정

단계	방법	데이터 규모	정확도
Step 1	IMDB 사람 라벨 데이터로 초기 학습	4 만 건	86.4%
Step 2	모델 예측 + 평점 이중검증으로 라벨 정제 후 재학습	43 만 건	98.0%
Step 3	명작 44 편 / 망작 43 편으로 파인튜닝	87 편	88.0%

에포크별 검증 정확도 추이(Step 3 기준): E1=0.8712, E2=0.8775, E3=0.8825, E4=0.8827, E5=0.8832

4.4 추론 방식

```
neg_probs = preds[:, 0] # softmax 부정 클래스 확률
labels[neg_probs >= 0.7] = -1 # 부정
labels[neg_probs <= 0.4] = 1 # 긍정
# 0.4 ~ 0.7 구간: 중립 (판단 보류)
```

한국어 리뷰와 60 단어 이하의 짧은 리뷰는 판단 신뢰도가 낮아 중립으로 처리했다.

5. 서비스 설계 및 성능 평가

5.1 추천 점수 공식

```
score = log(1 + 유사_리뷰_수) × 평균_평점
# 부정 비율 80% 이상인 영화는 추천에서 제외

# 예: Miracle in Cell No.7
# log(1 + 28) × 9.29 = 31.3
```

5.2 GRU 모델 성능 검증

검증셋	정확도
IMDB 전체 50,000 건	73.5%
단문 (5,946 건)	55.8%
극단 평점 — 긍정	90.4%
극단 평점 — 부정	1.1%

극단 평점 구간에서 긍정과 부정의 정확도 격차(90.4% vs 1.1%)가 매우 커서, 모델이 부정 리뷰를 제대로 걸러내지 못하는 경향이 뚜렷하게 나타났다.

5.3 추천 결과 검증

지표	값
혼동행렬	TP=414, FP=86
Precision	82.8%
쉬운 검색어 Recall	93.3%
어려운 검색어 Recall	22.9%

검색어의 난이도(구체성, 데이터 내 빈도)에 따라 Recall 편차가 매우 컸다(93.3% vs 22.9%). 어려운 검색어에서의 낮은 Recall 은 이 서비스의 핵심 한계로 이어진다.

2 부 · 개선 배경

기존 서비스의 한계를 발견하고 새로운 요구사항을 정의하는 과정

6. 한계 발견 및 방향 전환

6.1 발견된 한계

1 부 5.3 절의 검증 결과에서 나타나듯, GRU 감성 필터를 적용해도 추천 품질(어려운 검색어 Recall 22.9%)이 만족스럽지 않았다. 더 근본적인 문제는 감성 필터의 실질적 기여도였다: GRU 필터를 제거하고 벡터 유사도 검색만으로 추천을 생성해도 결과가 크게 다르지 않았다. 즉 GRU 는 서비스의 핵심 로직이 아니라 선택적으로 덧붙는 부품에 가까웠다.

6.2 교수 피드백

"GRU 감성 필터가 벡터 검색 결과와 실질적으로 중복된다"는 피드백을 받았다. 벡터 검색이 이미 의미적으로 유사한 리뷰를 찾아내므로, 감성 분류가 별도로 기여하는 정보량이 적다는 지적이었다.

6.3 새 요구사항

핵심 요구사항은 다음 한 문장으로 정리된다: "모델이 없으면 서비스 자체가 동작하지 않는 구조로 만들 것."

이 요구사항 아래 세 가지 방향을 검토했다.

검토 방향	내용	채택 여부
감성 점수 통합	기존 벡터 검색 점수에 감성 점수를 가중치로 결합	기각 — 여전히 검색이 주, 감성이 부수적
장르·무드 분류	리뷰로부터 영화의 장르·분위기를 분류하는 모델 학습	기각 — 키워드 매칭만으로 동일 결과 재현 가능
키워드 감성 검색	특정 키워드가 긍정/부정 중 어느 맥락으로 쓰였는지 판별	채택 — 키워드 매칭으로는 대체 불가능한 판단

키워드 감성 검색을 채택한 이유는, "scary"라는 단어가 들어간 리뷰라도 "So scary that I loved every moment"(긍정)와 "It tried to be scary but completely failed"(부정)처럼 전혀 다른 의미를 가질 수 있고, 이 판별은 키워드 매칭이나 평점만으로는 불가능해 모델이 실질적으로 필요한 구조가 되기 때문이다.

3 부 · 현재 서비스

LSTM 키워드 감성 추천

키워드가 긍정적으로 언급된 영화를 추천하는 새로운 서비스

7. 방향 전환 과정

2 부에서 정의한 요구사항에 따라, 서비스의 핵심 질문을 "이 영화가 이 검색어와 유사한가"에서 "이 키워드가 이 영화 리뷰에서 긍정적으로 언급되었는가"로 바꿨다. 이 판단은 리뷰 평점이나 키워드 존재 여부만으로는 할 수 없으므로, 감성 분류 모델이 서비스 결과에 직접적으로 반영되는 구조가 되었다.

예시: "scary"라는 단어가 들어간 리뷰라도,

- "So scary that I couldn't sleep for days, loved every moment" → 긍정 (0.732)
- "It tried to be scary but completely failed" → 부정 (0.038)

이 차이는 평점만으로는 잡을 수 없다. 8 점 리뷰도 특정 요소에 대해선 부정적일 수 있다.

8. LSTM 이진 분류기 (v1 → v3)

8.1 순환 레이어 선택

키워드 서비스에도 처음엔 1 부와 동일하게 GRU 를 사용해봤으나, 이번 과제에는 LSTM 이 더 적합하다고 판단해 LSTM 으로 교체했다.

8.2 레이블 노이즈 문제

처음 시도한 방법은 리뷰에서 키워드가 포함된 문장만 추출하고 그 리뷰의 전체 평점을 레이블로 쓰는 것이었다. 결과적으로 학습이 되지 않았다(loss 0.693 고착). 원인은 레이블 불일치 — 8 점 리뷰의 한 문장이 특정 요소에 대해서는 부정적일 수 있기 때문이다. 이 노이즈가 누적되면 모델이 패턴을 찾을 수 없다.

해결책: 문장 추출을 없애고 리뷰 전문(앞 500 자)과 전체 평점을 그대로 사용했다. 리뷰 전체의 감성은 평점과 일치하므로 레이블 신뢰도가 높다. 이 방식으로 84%까지 수렴했다.

8.3 모델 구조

```
Sequential([
  Embedding(30001, 64),
  SpatialDropout1D(0.15),
  LSTM(64),
  Dense(32, activation="relu"),
  Dropout(0.2),
  Dense(1, activation="sigmoid"),
])
# optimizer: Adam(lr=0.001)
# loss: binary_crossentropy
# MAX_LEN=80, VOCAB_SIZE=30000, BATCH=64
```

8.4 v1 → v2 → v3 변화

버전	데이터 규모	평점 기준	정확도	비고
v1	80k × 2	긍정 9-10 / 부정 1-2	84%	부정 편향 (recall 0.856 vs 0.746)
v2	80k × 2	긍정 8-9 / 부정 2-3	84%	균형 개선
v3	500k × 2	긍정 8-9 / 부정 2-3	84%	긍정/부정 균형(0.84/0.84) ← 배포

"scary"처럼 표면적으로 부정적인 단어도 학습 데이터가 30K 일 때는 "So scary that I couldn't sleep, exactly what I wanted"를 부정(0.277)으로 오분류했다. 데이터를 80K 로

놀리자 "loved", "wanted" 같은 긍정 신호가 "scary"를 override 하는 패턴을 학습해 동일 문장이 0.732(긍정)로 정상 분류됐다.

8.5 학습 결과

테스트셋 (16,000 건) 기준

클래스	precision	recall	f1-score
부정	0.85	0.83	0.84
긍정	0.83	0.85	0.84
전체	0.84	0.84	0.84

9. 키워드 인덱스 + Wilson 랭킹

9.1 사전 계산 인덱스

서비스 응답 속도를 위해 런타임 모델 추론을 제거했다. 리뷰 수 상위 영화들에 대해 키워드별 긍정/부정 문장 수를 오프라인에서 미리 계산해 pickle 로 저장하고, 서비스는 이 pkl 파일들을 로드해 Wilson 랭킹 계산과 필터링만 수행한다.

```
for movie in top_movies:
    # 키워드 포함 문장 추출 후 LSTM 배치 추론
    probs = model.predict(padded, batch_size=512)
    for (kw, sent), prob in zip(all_sentences, probs):
        if prob >= 0.5: stats[kw][movie]["positive"] += 1
        else: stats[kw][movie]["negative"] += 1
```

9.2 작은 표본이 랭킹 상위를 독점하는 문제

초기 정렬 기준은 단순 긍정 비율이었다. 3/3(100%)이 30/34(88%)보다 무조건 위로 가는 문제가 있었다.

긍정/전체	단순 비율	Wilson 하한
3/3	100%	0.438
10/10	100%	0.722
30/34	88%	0.734

```
def wilson_lower_bound(positive, total, z=1.96):
    if total == 0: return 0.0
    phat = positive / total
    denom = 1 + z*z/total
    center = phat + z*z/(2*total)
    margin = z * math.sqrt((phat*(1-phat) + z*z/(4*total)) / total)
    return (center - margin) / denom
```

Wilson 신뢰구간(95%) 하한으로 정렬 기준을 교체하자 30/34(88%)가 10/10(100%)보다 위로 가고, 3/3 은 하위로 밀렸다. 표본 크기가 자연스럽게 랭킹에 반영된다.

10. 서비스 설계

10.1 전체 아키텍처

- 사용자 인터페이스 계층: Streamlit 이 렌더링하는 장르 선택·키워드 클릭 UI
- 서비스 계층: KeywordService 가 인덱스를 조회하고 Wilson 랭킹을 적용해 결과를 반환
- 인덱스 계층: 사전 계산된 pkl 파일 — 서버 시작 시 한 번만 로드, 런타임 추론 없이 즉시 응답
- 오프라인 계층: 인덱스를 생성하기 위한 원본 데이터와 LSTM 모델 (재빌드 시에만 사용)

10.2 KeywordService

```
class KeywordService:
    def get_movies(self, keyword, genre=None, top_n=20, min_mentions=5):
        candidates = self.keyword_index[keyword]
        if genre:
            genre_movies = set(self.genre_map[genre]["movies"])
            candidates = {k: v for k, v in candidates.items() if k in genre_movies}
        ranked = []
        for k, v in candidates.items():
            if v["total"] < min_mentions: continue
            score = wilson_lower_bound(v["positive"], v["total"])
            ranked.append((k, v, score))
        ranked.sort(key=lambda x: x[2], reverse=True)
```

단일 키워드는 get_movies(), 복수 키워드는 get_movies_multi()를 호출한다. 복수 키워드의 경우 교집합만 남기고, 랭킹 점수는 각 키워드 Wilson 점수 중 최솟값을 사용한다.

11. 검증 및 한계

11.1 내부 검증

검증셋	정확도
내부 테스트셋 16,000 건	84%
IMDB 50,000 건 (v1)	80.1%
IMDB 50,000 건 (v2)	81.0%
짧은 리뷰 ≤80 단어 (v1)	88.5%
짧은 리뷰 ≤80 단어 (v2)	89.4%

예시 문장 8 건 전건 정답 — 리뷰 전문으로 학습한 모델이 문장 단위 판별에도 잘 일반화됨을 확인했다.

11.2 한계

추천 풀의 범위 제한

키워드 인덱스는 리뷰 수 기준 상위 일부 영화만 대상으로 사전 계산되므로, 리뷰 수는 적지만 키워드 감성이 강한 컬트작·독립 영화가 결과에서 누락될 수 있다.

문장 단위 일반화의 구조적 한계

모델이 리뷰 전문으로 학습되었기 때문에 긍정 신호가 희박한 짧은 문장에서는 오분류 가능성이 있다. 다만 서비스는 영화 전체에 걸쳐 긍정 비율을 집계하고 Wilson 하한으로 랭크하므로 개별 문장 오분류의 영향이 희석되고, 표본이 작은 영화는 자동으로 순위에서 밀려난다.

다의어로 인한 잔존 오분류

"scary"처럼 부정적 의미를 내포한 단어가 관용적으로 긍정 맥락에 쓰이는 경우, 장르 분류나 감성 판별에 잔여 오차가 남을 수 있다. 표본 크기가 작은 오분류 사례는 Wilson 점수가 낮아 상위 결과에는 노출되지 않는 방식으로 완화된다.

결론

두 서비스 비교

항목	기존 서비스 (1 부)	현재 서비스 (3 부)
핵심 구조	BGE-M3 임베딩 + FAISS 검색 + GRU 감성 필터	LSTM 키워드 감성 분류 + Wilson 랭킹
모델의 역할	선택적 — 제거해도 추천 결과 유지	필수 — 제거 시 서비스 핵심 판단 불가
감성 분류기	GRU, 3-클래스, 정확도 73.5% (IMDB 전체)	LSTM, 이진 분류, 정확도 84% (내부), 81.0%(IMDB, v2)
추천 Precision	82.8% (어려운 검색어 Recall 22.9%)	Wilson 하한 기반 랭킹으로 소표본 편향 제거
응답 방식	런타임 벡터 검색 + 모델 추론	오프라인 사전 계산 + pkl 조회 (수십 ms)

개발 과정에서 얻은 교훈

- 모델의 실질적 기여도 검증: 모델을 제거했을 때 결과가 달라지지 않는다면, 그 모델은 서비스의 핵심이 아니라 장식에 가깝다. 이 판단 기준이 1 부에서 3 부로의 전환을 이끌었다
- 순환 레이어 선택: 3 부에서는 GRU 대신 LSTM 을 사용해봤다. 과제 특성에 맞는 레이어를 실험적으로 선택하는 과정이 필요했다
- 레이블 설계: 문장 단위 추출 후 전체 평점을 레이블로 쓰면 노이즈가 심해 학습이 수렴하지 않는다. 리뷰 전문 + 전체 평점 조합이 안정적
- 순위 알고리즘: 단순 비율/점수 정렬은 소표본 편향에 취약하다. Wilson 하한처럼 표본 크기를 반영하는 지표로 해결 가능

향후 개선 방향

- 추천 풀 확대: 사전 계산 대상 영화 수를 늘리거나, 리뷰 수가 적어도 Wilson 점수가 충분한 영화를 포함하는 방식 검토
- 다중 레이블 감성: 현재 단일 긍/부정 이진 분류를 다중 클래스로 확장해 세밀한 분류 가능
- 키워드 확장: 현재 키워드 목록을 자동 발굴 알고리즘으로 확장 (TF-IDF 기반 특징어 추출 등)

- 다국어 지원: 현재 영문 리뷰 중심 구조를 다국어로 확장 (한국어 리뷰 별도 모델)